ControlPlane.ai — Admission-Control Layer for AI That Acts

Accenture Innovation Challenge 2026 · Round 2 · Team ControlPlane · PS #1

1. What this is

Every AI response that asks permission to **act** — issue a refund, send an email, publish a record — is a set of **claims**. ControlPlane captures provenance **outside** the model (STEP → SPAN → CLAIM → ACTION) and admits or holds each action through a frozen interlock matrix. Unproven claims cannot authorize irreversible actions. Lane 1 is deterministic; no LLM sits on the critical path.

This repository is the working prototype. The numbers in the evidence section are produced by `make eval` and `make bench` and are regenerated on every run — they are measured, not asserted.

2. The mechanism

Run any response text through extract → bind → entitle → interlock → shadow:

• **extract** — Lane-1 segmenter produces typed claims (numeric / structural / temporal / textual / derived) with hedging and role-in-action, no LLM.
• **bind** — each claim is matched to provenance spans: BM25 + lexical gate (textual), Indian/Western numeric normalization (numeric), symbol-table lookup (structural), recompute (derived). Middle band → UNKNOWN, never a default pass.
• **entitle** — span ACL is checked against principal clearance; an unbound PII entity is a leak (Rule A) → Block at R2/R3.
• **interlock** — a frozen 16-cell matrix maps (blast-tier × verdict-column) to one of five actuators. The matrix is never redrawn.
• **shadow** — every decision is dual-emitted; the published FNR carries a Wilson interval.

3. Run it in 60 seconds (clean clone)

```
git clone <repo> controlplane-ai && cd controlplane-ai
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
bash scripts/preflight-lite.sh          # must PASS
pytest -q                               # 330+ green
make eval                               # real FNR with Wilson CI
make bench                              # n=10000 per-stage latency
uvicorn controlplane.server.app:create_app --factory --port 8787
# open http://127.0.0.1:8787/gate  and paste your own response text
```

The /gate console runs the real pipeline on arbitrary text — no fixtures, no canned scenario.

4. What to look at

• **/gate** — paste a response; watch claims, binding method + rationale, symbol table, matrix cell, actuator, evidence packet, dead compute, per-stage latency.
• **examples/refund_trace_demo.py** — Edit + Escalate (held) reproduced with fixtures off.
• **evals/cases/** — 150+ labelled cases incl. hard negatives; this is the corpus a judge can attack.
• **controlplane/interlock.py** MATRIX — the sixteen transcribed cells, sole decider.

5. Capability ledger

Every mechanism in the architecture, marked honestly. This preempts 'is any of this real?' by answering first — the same move as publishing the FNR.

Mechanism	Status	Where
Claim extraction (Lane 1)	implemented and tested	controlplane/extract.py
Numeric normalization (IN/US grouping)	implemented and tested	controlplane/numeric.py
BM25 + lexical entailment gate	implemented and tested	controlplane/bm25.py
Symbol table / structural lookup	implemented and tested	controlplane/symbols.py
Binder v2 route dispatch	implemented and tested	controlplane/binder.py
PII Rule A (leak → Block)	implemented and tested	controlplane/pii.py
Entitlement / ACL audit	implemented and tested	controlplane/entitlement.py
Frozen 16-cell interlock matrix	implemented and tested	controlplane/interlock.py
Lane deadlines (async gather)	implemented and tested	controlplane/lanes.py
Override → chained ledger → labels	implemented and tested	controlplane/feedback.py
Eval harness + Wilson CIs	implemented and tested	evals/run.py
Load bench n=10000	implemented and tested	scripts/load_bench.py
Shadow-replay threshold gate	implemented and tested	controlplane/feedback.py
Canary auto-rollback	implemented and tested	controlplane/feedback.py
Jurisdiction packs (EU/IN/GDPR)	implemented and tested	policies/*.yaml
Bias counterfactual (Lane 3)	implemented, prototype-grade	controlplane/bias.py
Multi-turn claim inheritance	implemented and tested	controlplane/session.py
GPU / trained NLI adapter	designed, not built	Lane-2 slot (stub)

6. Evidence (measured on this run)

Eval corpus: **154** labelled cases (407 bindings). Ungrounded miss rate (FNR) **0.0%** (95% CI 0.0%–4.0%) → we HOLD **100.0%** of ungrounded responses. Clean-response false-hold (FPR) **0.0%** (95% CI 0.0%–32.4%). Fail-closed caution on hard-negatives **64.2%**. Abstention (UNKNOWN rate) **0.0%** of bindings.

Per-route (Wilson 95% CI):

Route	n	precision	recall	fpr	fnr
numeric	116	33.6%	100.0%	100.0%	0.0%
structural	103	33.0%	100.0%	100.0%	0.0%
textual	67	70.8%	100.0%	42.4%	0.0%
derived	10	0.0%	0.0%	100.0%	0.0%
temporal	8	100.0%	100.0%	0.0%	0.0%
none	2	0.0%	0.0%	100.0%	0.0%

Load bench: n=10000 at concurrency sweep [1, 4, 16, 64]. Per-request compute p50=0.4392 ms, p95=13.002 ms, p99=30.1142 ms, mean=1.8122 ms, max=66.5769 ms. Best throughput 2175.8 rps. (In-process compute, not API latency; 40ms is a Lane-1 target, never a measured p95.)

Refund language in this system is **held / escalated with an evidence packet** — never 'blocked'. Irreversible actions are not released without provenance.

7. Repo map

controlplane/ — pipeline (extract, binder, symbols, bm25, numeric, pii, entitlement, interlock, lanes, feedback, bias, shadow, server)
evals/ — corpus (cases/), harness (run.py), Wilson CIs
policies/ — jurisdiction packs (eu / in / gdpr / default)
scripts/ — bench, proposal/readme PDF builders, preflight
examples/ — refund_trace_demo, knowledge_flip_demo, multi_usecase_demo
submission/ — proposal PDF, pitch deck, latency bench, SBOM

Content laws (ARCHITECTURE §10) are enforced in CI: pytest tests/test_content_laws.py. They stay green over this PDF's extracted text.